

MagicQ miniQ Encoders

A custom hardware controller for the virtual encoders and windows of **ChamSys MagicQ**, built from a **Raspberry Pi Pico** (RP2040).

The Pico emulates a native **keyboard + absolute mouse + relative mouse** (composite HID) over USB, so no additional drivers need to be installed. Every physical knob first moves the mouse cursor onto its matching **on-screen encoder** in MagicQ and then scrolls or clicks exactly there — no key codes are typed and no manual cursor positioning is needed.

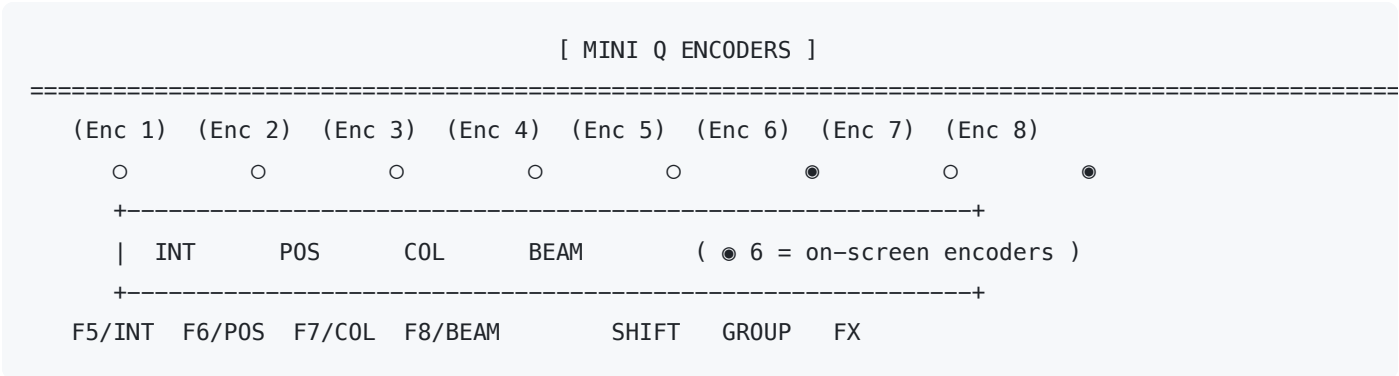
Note: The firmware works in **any** MagicQ keyboard mode, including **Playback shortcuts** (which the Fader wing needs). No CAPS LOCK / mode toggling is used.

Table of contents

- [1. The layout \(physical arrangement\)](#)
 - [1.1 Control mapping \(verified\)](#)
 - [1.2 Cursor position calibration \(no re-flashing needed\)](#)
- [2. Required hardware & shopping list](#)
- [3. Hardware wiring](#)
 - [3.1 Encoder direct connection \(Pico GPIO\)](#)
 - [3.2 Direct keys on the Pico \(GPIO, against GND\)](#)
 - [3.3 Keys on the MCP23017 \(I/O expander, address 0x20 \)](#)
 - [3.4 Connection diagrams \(ASCII\)](#)
- [4. Build & upload \(PlatformIO\)](#)
- [5. Setup in MagicQ](#)
- [Notes & limitations](#)
- [Project structure](#)

1. The layout (physical arrangement)

The console has **8 rotary encoders with click** plus **7 direct keys** below them:



- **Enc 1–8** are incremental rotary encoders (CLK/DT) with a push button (SW) on top of the shaft.
- The **F5–F8, Shift, Group and FX** keys switch directly against **GND** on Pico GPIOs.

- Only the **8 encoder SW** sit on the **MCP23017** I/O expander (I²C) — that is what keeps the 8 clicks plus the direct keys within the Pico's GPIO budget.

1.1 Control mapping (verified)

Control	Action	HID output
Encoder right	Cursor to on-screen encoder + wheel up (value +)	abs. mouse + wheel +1
Encoder left	Cursor to on-screen encoder + wheel down (value −)	abs. mouse + wheel −1
Encoder click (SW)	Cursor to on-screen encoder + click	abs. mouse + left click
Shift (method A)	Ultra fine adjustment (0.1 % steps)	KEY_LEFT_SHIFT (held)
Intensity (F5)	Open INT window	Ctrl+I
Position (F6)	Open POS window	Ctrl+P
Colour (F7)	Open COL window	Ctrl+K
Beam (F8)	Open BEAM window	Ctrl+J
Group	Open Group window	Ctrl+G
FX	Open FX window	Ctrl+F

All buttons and switches switch directly against **GND** (pull-ups enabled).

How the encoders work: MagicQ on PC/Mac shows its **on-screen encoders** in the console layout (full screen / maximized). The 8 physical knobs first move the **mouse cursor** to the matching on-screen encoder (absolute positioning, see `encCal.x/y`) and then send a `**mouse wheel**` scroll or left click exactly there.

Important: The attribute keys (F5–F8), Group and FX send **Ctrl combinations** (`Ctrl+I`, `Ctrl+P`, `Ctrl+K`, `Ctrl+J`, `Ctrl+G`, `Ctrl+F`) and thus open the corresponding MagicQ windows directly. The **Shift key** holds `KEY_LEFT_SHIFT` as a modifier for ultra fine adjustment.

1.2 Cursor position calibration (no re-flashing needed)

Each of the 8 on-screen encoders has its **own** target position (no interpolation), so even an uneven MagicQ layout is hit exactly. The values live in `encCal.x[8]` / `encCal.y[8]` (`src/main.cpp`) and are stored permanently in the Pico's LittleFS flash as `/ecal.bin`.

Restart the Pico while holding down encoder 1 (its switch) → the calibration UI starts automatically. The mouse cursor follows the selected encoder, so you can see what you are moving.

Action	Function
<code>**Click encoder *i**</code>	select encoder <i>i</i> , cursor jumps to its stored position

Action	Function
Turn encoder 1 / 2	selected encoder: X coarse / fine (± 500 / ± 50)
Turn encoder 3 / 4	selected encoder: Y coarse / fine (± 500 / ± 50)
Click encoder <i>i</i> again	save that encoder's position — the LED blinks the number of encoders saved so far
all 8 saved	<code>/ecal.bin</code> is written and the Pico restarts automatically — this is the only way out of calibration

Turn clockwise = plus (right / down). Encoders 1–4 act purely as adjustment knobs while calibrating; the target positions always come from the encoder SW clicks. `[CAL] ...` on the serial console reports every selection, adjustment and save.

Note: The calibration values are stored in LittleFS, which requires the ****64 KB filesystem partition**** (`board_build.filesystem_size = 64` in `platformio.ini`). Without it `LittleFS.begin()` fails, nothing is saved and the firmware prints a warning. Values are read back on every boot (`[CAL] /ecal.bin geladen: ...`, otherwise the compiled-in defaults).

For reference the old blink-through mode still exists: set `#define ENC_CALIBRATE 1` in `src/main.cpp` to walk through all 8 target points (4 s each); the values are printed on the serial console and can then be entered as defaults.

2. Required hardware & shopping list

Qty	Component	Note
1	Raspberry Pi Pico (RP2040)	with Micro-USB
1	MCP23017 I/O expander (DIP-28)	only for the 8 encoder SW
8	Mech. rotary encoders with push button (e.g. EC11)	20 detents, 3-pin variant
4	Buttons (momentary)	F5–F8 (directly on the Pico)
1	Button or toggle switch	Shift (directly on the Pico, GP22)
2	Buttons (momentary)	Group, FX (directly on the Pico)
2	Resistors 2.2 k Ω (optional)	pull-up for SDA/SCL
1	Resistor 10 k Ω (recommended)	pull-up for the MCP23017 RESET
-	hook-up wires / stranded wire	for wiring
-	optional: pin headers / breadboards	

Pico pin usage (25 of 27 GPIOs):

- **GP0–GP15** = 8 encoders (CLK/DT)
- **GP16/GP17** = I²C for the MCP23017

- **GP18–GP21** = F5–F8
- **GP22** = Shift
- **GP26** = Group, **GP27** = FX

The **MCP23017** only takes the **8 encoder SW**. **GP28** remains as a free reserve.

3. Hardware wiring

3.1 Encoder direct connection (Pico GPIO)

Component	CLK / DT
Encoder 1	GP0 / GP1
Encoder 2	GP2 / GP3
Encoder 3	GP4 / GP5
Encoder 4	GP6 / GP7
Encoder 5	GP8 / GP9
Encoder 6	GP10 / GP11
Encoder 7	GP12 / GP13
Encoder 8	GP14 / GP15

3.2 Direct keys on the Pico (GPIO, against GND)

Function	Pico pin	Shortcut
Intensity (F5)	GP18	Ctrl+I
Position (F6)	GP19	Ctrl+P
Colour (F7)	GP20	Ctrl+K
Beam (F8)	GP21	Ctrl+J
Shift (method A)	GP22	held KEY_LEFT_SHIFT
Group	GP26	Ctrl+G
FX	GP27	Ctrl+F

3.3 Keys on the MCP23017 (I/O expander, address 0x20)

The MCP23017 is connected to the Pico via I²C (address 0x20). It only takes the **encoder SW** (8 of 16 pins); all switches go against GND (pull-up in the MCP enabled).

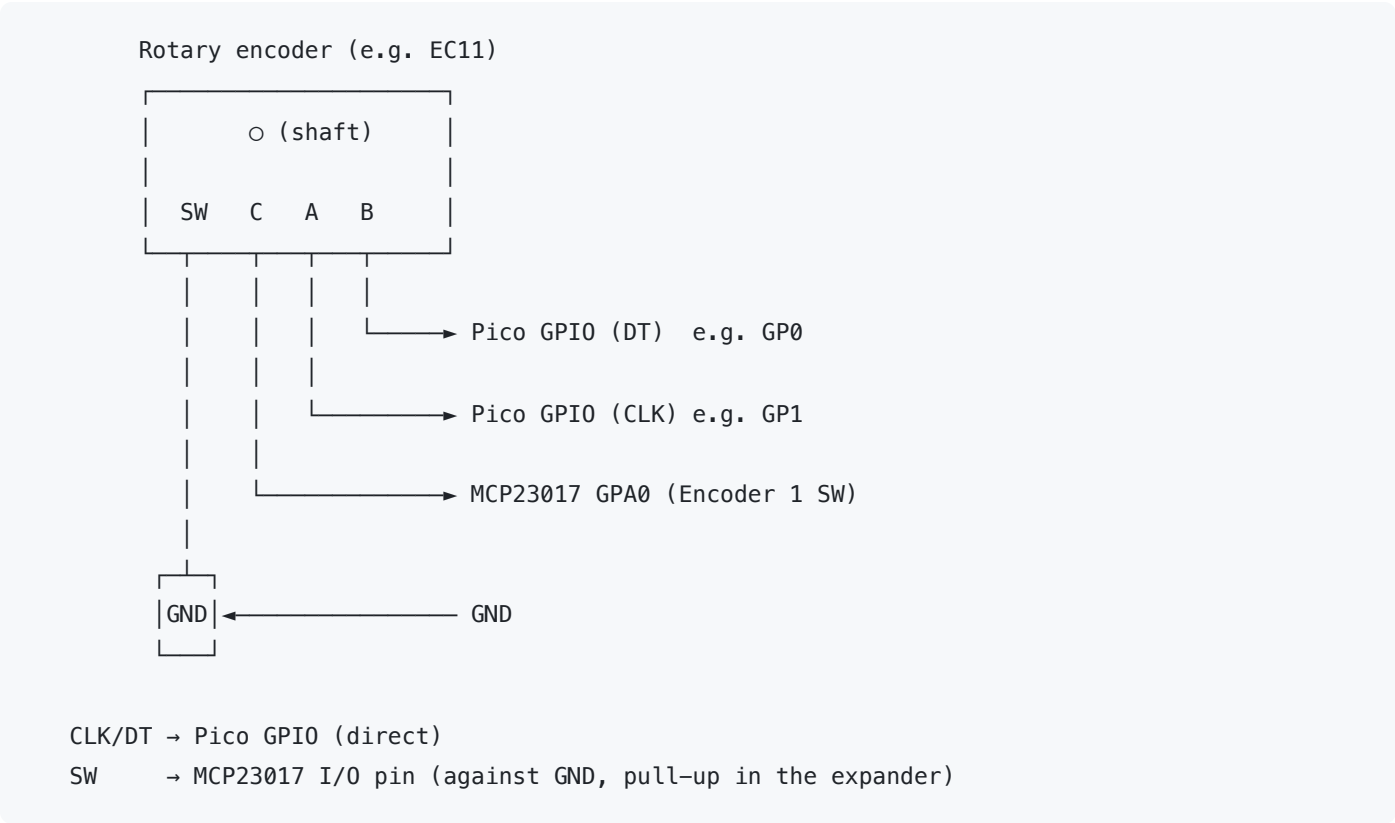
Expander pin	Function	HID output
GPA0 (pin 21)	Encoder 1 SW	left click
GPA1 (pin 22)	Encoder 2 SW	left click
GPA2 (pin 23)	Encoder 3 SW	left click
GPA3 (pin 24)	Encoder 4 SW	left click
GPA4 (pin 25)	Encoder 5 SW	left click
GPA5 (pin 26)	Encoder 6 SW	left click
GPA6 (pin 27)	Encoder 7 SW	left click
GPA7 (pin 28)	Encoder 8 SW	left click
GPB0–GPB7	free	–

The MCP23017 is configured with pull-ups; each key switches an expander pin against **GND**.

3.4 Connection diagrams (ASCII)

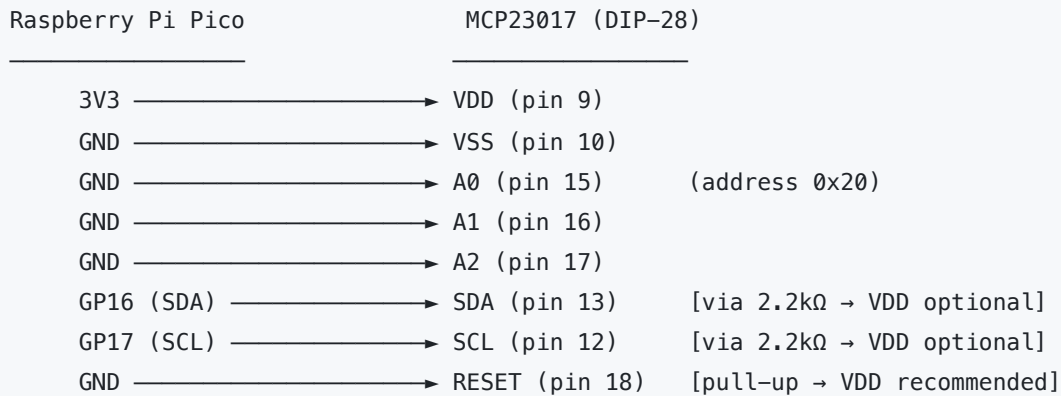
Rotary encoder (CLK / DT): incremental rotary encoder with integrated push button (SW).

CLK/DT go directly to the Pico, **SW** (common) to the MCP23017:



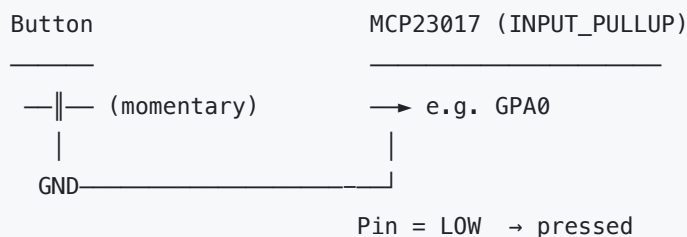
For encoders with a common connection (C), connect **C to GND**; A/B (CLK/DT) to the Pico GPIOs, the push button (SW) is internally switched against C and goes to the MCP23017.

MCP23017 I/O expander (encoder SW only), Pico ↔ MCP23017 via I²C:

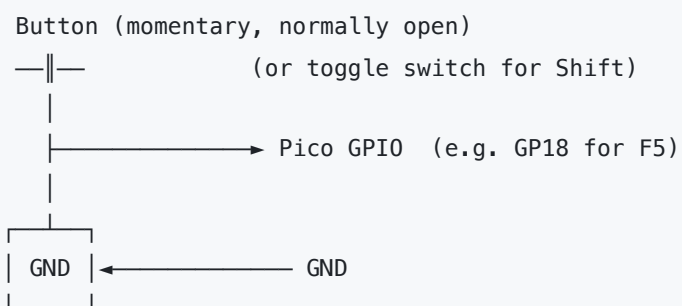


Keys on the MCP23017 (switch against GND):

Encoder-SW 1..8 → GPA0..GPA7 (pins 21..28)



Direct keys on the Pico (F5–F8, Shift, Group, FX) — all switch against GND
(F5=GP18, F6=GP19, F7=GP20, F8=GP21, Shift=GP22, Group=GP26, FX=GP27):



Software: `pinMode(gpio, INPUT_PULLUP)`

GPIO = LOW → key pressed

GPIO = HIGH → key released

4. Build & upload (PlatformIO)

Prerequisite: [PlatformIO Core](#) installed.

```
# Switch to the project directory
cd Enncoder
```

```
# Compile the firmware
pio run

# Compile + upload to the Pico (hold the BOOTSEL button before plugging in)
pio run -t upload

# Serial console (debug output)
pio device monitor
```

Important: For upload with the earlephilhower core, the Pico normally only needs to be put into bootloader mode via **BOOTSEL** (hold, plug in USB, release) the first time. Later uploads work directly over USB.

Note on libraries: The USB keyboard (`Keyboard`) comes from the earlephilhower core itself. The `lib_ldf_mode = chain+` in `platformio.ini` ensures that the required core library `tusb-hid` is built automatically. **No** external TinyUSB package is needed.

`board_build.filesystem_size = 64` reserves the 64 KB **LittleFS** partition that the encoder position calibration needs (section 1.2).

5. Setup in MagicQ

The absolute-mouse encoder control works in **any** MagicQ keyboard mode, no mode switch is required. The F5–F8 / Group / FX window shortcuts work best in **Programming shortcuts** or `CTRL + Keys for Windows` mode:

1. Navigate to: **Setup > View Settings > Keypad Encoders**.
2. Set the option **MagicQ PC Keyboard Mode** to **Programming shortcuts**

(or `CTRL + Keys for Windows`).

1. Start MagicQ **full screen / maximized** so the on-screen encoders of the console layout appear at fixed screen positions.

1. If MagicQ runs maximized on another resolution or window size, run the position calibration once (section 1.2) — the firmware then works with the layout actually in use.

Then every turn/click of a Pico knob moves the cursor to the matching on-screen encoder and changes that attribute. If you prefer to keep MagicQ in **Playback shortcuts** (e.g. for the Fader wing), the encoders still work — only the attribute window keys (F5–F8/Group/FX) need the mode above.

Notes & limitations

- **Almost everything directly on the Pico:** The **F keys (F5–F8, GP18–GP21), Shift (GP22), Group (GP26) and FX

(GP27) **are connected** directly to the Pico. **Only the 8 encoder SW sit on the single MCP23017****, so that all 8 encoders including the click function fit.

- **Reserve pin: GP28** stays free – e.g. another key (e.g. Layout 1)

could be connected there.

- **Layout keys (Lay 1–3):** These were removed. If desired, they can easily be connected to

GP28 or to the free MCP23017 pins (GPB0–GPB7) and added in the code.

- **Absolute-mouse encoders:** The 8 knobs work as an **absolute mouse** (positioning + wheel) plus a **relative mouse** for the click (composite HID keyboard+absolute+relative mouse). Turning/clicking a knob first positions the cursor on the matching on-screen encoder (`encCal.x/y`) and then scrolls/clicks there. Positioning and the wheel go through the absolute report; the click uses the relative mouse, because macOS does not treat the digitizer-style absolute buttons as a left click. Works in every keyboard mode and needs no CAPS LOCK / mode toggling. Positions are calibrated at runtime and stored in `/ecal.bin` (see section 1.2).

- **Ctrl combinations:** F5–F8 (INT/POS/COL/BEAM), Group and FX send `Ctrl+<key>` and thus open the respective MagicQ windows directly (needs `Programming shortcuts` or ``CTRL + Keys for Windows`` keyboard mode).

- **USB-HID:** The Pico appears to the PC as a **keyboard + absolute mouse + relative mouse** (composite HID). The absolute mouse is registered **before the keyboard reports** — macOS otherwise ignores pointing devices in a composite. The relative mouse only handles the click button. Key presses / cursor moves / clicks are only sent after the USB-HID device has been mounted.

- **Shift keep-alive:** While the Shift key is held, the firmware re-sends the Shift-down report every 60 ms (`SHIFT_KEEPLIVE_MS`). macOS/MagicQ would otherwise treat a single down event as a "tap" and drop the modifier.

- **MCP23017 guard:** If the expander is missing or unreadable at boot, the firmware logs `[WARN] MCP23017 nicht erkannt` and disables only the encoder clicks (`mcpOK=false`) – rotation and all direct keys keep working instead of crashing on a missing I²C device. For the same reason the position calibration requires the expander and is skipped without it.

- **I²C address:** The MCP23017 is set to address `0x20` with A0/A1/A2 = GND (standard).
- The **serial output (115200)** is only for debugging and is not required for operation.

Project structure

```
Enncoder/
├─ README.md           # This handbook
├─ README.pdf          # PDF export of this handbook
├─ platformio.ini      # PlatformIO configuration
├─ pcb/                # Gerbers, BOM and pick&place for the encoder PCB
```

```
|— pcb 2/          # earlier PCB revision
|— src/
  |— main.cpp      # Firmware for the Raspberry Pi Pico
```